CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment – 2

Name: Aariz Raza
UID: 24BAI71000
Branch: CSE AIML (CS221)
Section/Group: 24AIT_NTPP-3[B]
Semester: 5th
Date of Performance: 08/07/2026
Subject: Full Stack-II
Subject Code: 24CSP-337

EXPERIMENT – 2.1

1. **AIM:** To design and implement a centralized state management system using Redux Toolkit for managing posts and platform-related data.

2. **OBJECTIVE:**

- To understand the concept of global state management.
- To implement Redux Toolkit for scalable state handling.
- To design a normalized state structure.
- To manage asynchronous data flows (optionally using mock APIs).

3. **SOFTWARE REQUIREMENTS:**

1. React.js
2. Node.js (LTS Version)
3. NPM (Node Package Manager)
4. Visual Studio Code
5. Redux Toolkit
6. React-Redux

4. **THEORY**

Redux Toolkit is the official and recommended library for managing application state in React. Unlike local component state managed by `useState()`, Redux Toolkit stores application data in a centralized global store, making it accessible to all components. This approach reduces code duplication, improves scalability, and simplifies state management in large applications.
In this experiment, Redux Toolkit is used to manage social media posts and platform-related data. A store is created to hold the application's global state, while a slice defines the state, reducers, and actions required to update the data. Components interact with the store using the `useSelector()` Hook to read data and the `useDispatch()` Hook to send actions that update the state. Redux Toolkit also simplifies configuration by providing the `configureStore()` and `createSlice()` functions, reducing boilerplate code compared to traditional Redux. This centralized approach makes the application easier to maintain and extend. The experiment introduces students to global state management, reducers, actions, slices, selectors, dispatch functions, and efficient data sharing, which are essential concepts for building scalable frontend applications.

5. IMPLEMENTATION/PROCEDURE

1. Install Node.js and create a React project.
2. Install Redux Toolkit and React-Redux using npm.
3. Create a Redux store using `configureStore()`.
4. Create a slice using `createSlice()` to manage posts.
5. Wrap the application with the `Provider` component.
6. Access global state using `useSelector()`.
7. Update state using `useDispatch()`.
8. Display the list of posts.
9. Test the add post functionality.
10. Run the application using `npm start`.

6. CODE:

```
import { useState } from "react";
import { useSelector, useDispatch } from "react-redux";
import { addPost } from "./store";

function App() {
 const [post, setPost] = useState("");

 const posts = useSelector((state) => state.posts.posts);
 const dispatch = useDispatch();

 const handleAdd = () => {
  if (post.trim() === "") return;

  dispatch(addPost(post));
  setPost("");
 };

 return (
  <div style={{ textAlign: "center", marginTop: "40px" }}>
   <h1>Redux Toolkit Demo</h1>

   <input
    type="text"
    placeholder="Enter Post"
    value={post}
    onChange={(e) => setPost(e.target.value)}
   />

   <button onClick={handleAdd}>Add Post</button>

   <h3>Posts</h3>

   <ul>
```

```
        {posts.map((item, index) => (
          <li key={index}>{item}</li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

7. OUTPUT

The application successfully implements centralized state management using Redux Toolkit. Users can enter a post and click Add Post to store it in the global Redux store. The saved posts are displayed dynamically, and every component can access the same data without passing props, making the application scalable and easier to maintain.

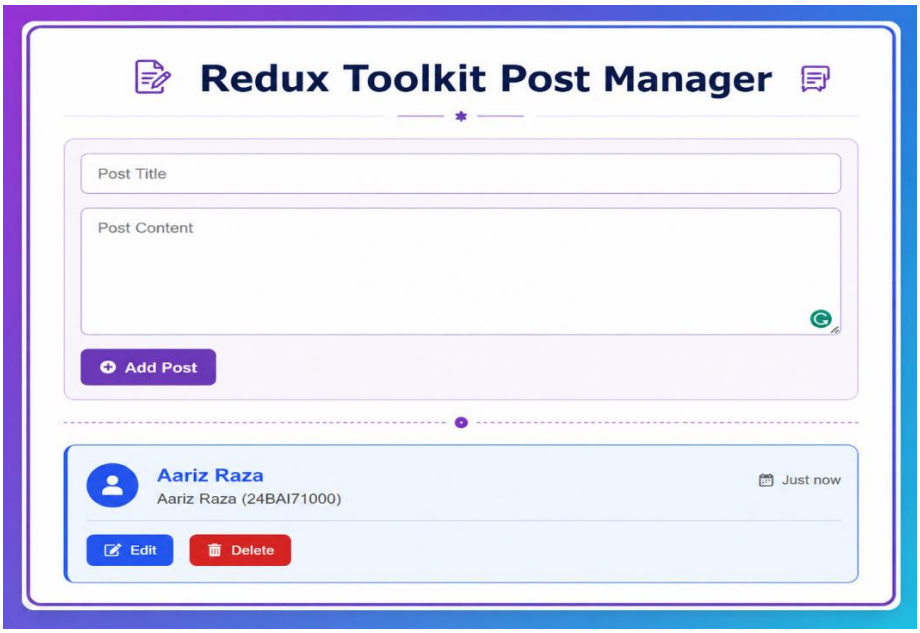

Fig 2.1: Redux Toolkit post manager

8. LEARNING OUTCOME

- Understand the concept of global state management.
- Learn the architecture of Redux Toolkit.
- Create and configure a Redux store.
- Implement state management using `createSlice()`.
- Access global state using `useSelector()`.
- Update state using `useDispatch()`.
- Build scalable React applications with centralized state.
- Improve code organization and maintainability using Redux Toolkit.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 2.2

1. AIM:

To optimize state access and improve application performance using memoized selectors and efficient rendering strategies.

2. OBJECTIVE:

- To understand the concept of memoization in React.
- To optimize component rendering using `React.memo()`.
- To improve application performance using `useMemo()`.
- To avoid unnecessary re-rendering of components.

3. SOFTWARE REQUIREMENTS:

1. Operating System: Windows/Linux/macOS
2. Node.js (LTS Version)
3. npm (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js
7. Command Prompt / Terminal

4. THEORY

Modern React applications often contain multiple components that re-render whenever the parent component updates. Unnecessary rendering can reduce application performance, especially when dealing with large datasets or complex user interfaces. To solve this problem, React provides memoization techniques that optimize rendering by preventing components from re-rendering unless their data changes.

In this experiment, `React.memo()` and `useMemo()` are used to improve application performance. The `useMemo()` Hook stores the result of a calculation or data in memory and recomputes it only when its dependencies change. This reduces repeated calculations during rendering. The `React.memo()` function memoizes a component so that it re-renders only when its props change. A simple student list is displayed along with a counter. When the counter value changes, only the counter component updates, while the student list remains unchanged because it is memoized. This reduces unnecessary rendering and improves efficiency.

This experiment helps students understand state optimization, memoization, efficient rendering, React Hooks, component reusability, and performance optimization, which are important concepts in modern React application development.

5. IMPLEMENTATION/PROCEDURE

1. Install Node.js and create a React project.
2. Open the project in Visual Studio Code.
3. Import useState, useMemo, and React.memo.
4. Create a counter using useState().
5. Create a student list using useMemo().
6. Create a memoized child component using React.memo().
7. Display the student list and counter.
8. Update the counter and observe that the student list does not re-render.
9. Run the application using npm start.
10. Verify the optimized rendering behavior.

11. CODE

```
import React, { useState, useMemo } from "react";

// Memoized Child Component
const StudentList = React.memo(({ students }) => {
  console.log("Student List Rendered");

  return (
    <div>
      <h3>Student List</h3>
      <ul>
        {students.map((student, index) => (
          <li key={index}>{student}</li>
        ))}
      </ul>
    </div>
  );
});

function App() {
  const [count, setCount] = useState(0);

  // Memoized Student Data
  const students = useMemo(() => {
    return ["Aariz", "Rahul", "Priya", "Neha"];
  }, []);

  return (
    <div style={{ textAlign: "center", marginTop: "40px" }}>
```

```
      <h1>Memoized State Example</h1>

      <h2>Counter: {count}</h2>

      <button onClick={() => setCount(count + 1)}>
        Increase Counter
      </button>

      <hr />

      <StudentList students={students} />
    </div>
  );
}

export default App;
```

12. OUTPUT (SCREENSHOT OF WEB + explain)

The application displays a counter and a list of students. When the Increase Counter button is clicked, only the counter value is updated. The student list does not re-render because it is memoized using React.memo(), and the student data is stored using useMemo(). This improves application performance by avoiding unnecessary rendering.

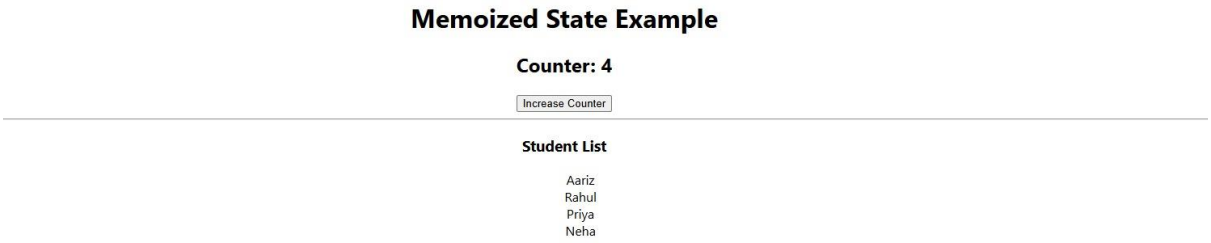

Fig 2.2: Memoized State Example

13. LEARNING OUTCOME

- Understand the concept of memoization in React.
- Learn the purpose of `React.memo()`.
- Implement `useMemo()` for optimizing state access.
- Improve application performance through efficient rendering.
- Prevent unnecessary component re-rendering.
- Understand the relationship between state and rendering.
- Build reusable and optimized React components.
- Develop efficient frontend applications using React optimization techniques.